

MIAE 215 Midterm Study Guide: Types, Operators, and Loops Premium

Course: Mechanical, Industrial & Aerospace engineering (MIAE 215) 378 documents

University: Concordia University

Hatim

2025/2026

Download

7

0

Save

↑

i

This document provides a comprehensive overview of programming concepts, including data types, operators, variable update: ...more

MIAE 215

A) TYPES & RANGES

int: -2,147,483,648..2,147,483,647 (overflow wraps); **bool**: 0 or 1; **char** often -128..127 (signed). **Float**: 1.2e-38 to 3.4e+38 (~8 sig. digits), **double** (~16 sig. digits) 2.3e-308 to 1.7e+308. **sizeof**(T) gives bytes. Uninitialized variables/array slots contain garbage.

Casting: (int)x truncates toward 0. **Modifiers**: short, long, unsigned, const.

B) OPERATORS & PRECEDENCE (highest → Lowest)

() ! */% +- < <= > >= == != && || = Integer division truncates (1/3==0); force double math (1/3.0 or (double)1/3).

Mixed types promote, but int/int happens before promotion. 1/3*10.0 → 0.0.

++i (change then use) vs i++ (use then change).

Short-circuit: a&& b stops if a is false; a||b stops if a is true.

C) VARIABLE UPDATES (COMMON IDIOMS)

i=i+1, i++, ++i are all +1; i+=k adds k; sum+=A[i];

Swap (tmp): tmp=a; a=b; b=tmp;

Min/Max update: if(x<mn) mn=x; if(x>mx) mx=x;

D) IF / ELSE IF / ELSE – SUPER-CLEAR REFRESHER

if = “Take this lane only if this sign matches.”
else if = “If the first sign didn’t match, check this next sign.”

else = “None of the signs matched? Take the default lane.” Use braces even for one-line bodies to avoid bugs. Combine conditions with && and ||, and group with ().

E) LOOPS (FOR / WHILE / DO-WHILE)

for(init; test; update){body} – **order**: init → test → body → update → test... (exit when test is false).

while(test){body} – use when iteration count unknown; ensure something changes in the loop.
do{body} **while**(test); – executes body at least once.

break exits the loop; **continue** jumps to update/test for next pass.

After loop, the index equals the first value that failed the test (useful in output questions).

F) ARRAYS (1D & 2D) – PATTERNS YOU’LL NEED

1D: double A[100]; indices 0..99. **Safe input**:
for(i=0; i<n && i<100; i++) cin>>A[i];

Init shortcuts: int Z[5]={0}; double B[]={1.1,2.2};

2D: double M[3][4]; int i, j; **for**(i=1; i<=3; i++) {
 for(j=1; j<=4; j++) {
 cin >> B[i][j];
 }
}

Dot product: sum=0; **for**(i=0; i<n; i++) sum += A[i]*B[i]; **Count range**: if(B[i]>0 && B[i]<PI) cnt++;

Mean & sample variance (no pow): ave = sum/n; then d=A[i]-ave; s2 += d*d; result = s2/(n-1).

Reverse: **for**(i=0; i<n; i++) R[i]=A[n-1-i]; **Merge**:

C[0..2n-1] from A[0..n-1] & B[0..n-1];
Find max in 2D: mx=M[0][0];
if(M[i][j]>mx) mx=M[i][j];

G) OUTPUT-BY-HAND

Mark executed lines; track only variables that change (compact table). At each cout, immediately write exactly what prints. Watch for: int truncation, i++ vs ++i, break/continue, array bounds, uninitialized (garbage).

For loops, remember the exact order (see E). Record the final index value used after a loop.

H) PROGRAMMING QUESTION – 7-STEP RECIPE

1) Declare 2) Initialize 3) Input 4) Control (if/loops) 5) Expressions 6) Output 7) Quick test

I) CODE SNIPPETS (DROP-IN)

// Dot product + count in (0,PI)

```
int i,n=10,cnt=0; double
A[10],B[10],sum=0,PI=4*atan(1.0);
for(i=0;i<n;i++){ cin>>A[i]; } // read A
for(i=0;i<n;i++){ cin>>B[i]; } // read B
for(i=0;i<n;i++){ sum+=A[i]*B[i]; } // dot
for(i=0;i<n;i++){ if(B[i]>0 && B[i]<PI) cnt++; }
// count
```

```
cout<<sum<<" "<<cnt<<"\n";
```

// Read ints until negative; then count & sum

```
int k,count=0,isum=0;
while(1){ cin>>k; if(k<0) break; count++;
isum+=k; }
cout<<count<<" "<<isum<<"\n";
```

// Mean & sample variance (n>1), no pow

```
int nA,i; double A2[100],sumA=0,ave,s2=0;
cin>>nA;
for(i=0;i<nA;i++){ cin>>A2[i]; sumA+=A2[i]; }
ave=sumA/nA;
for(i=0;i<nA;i++){ double d=A2[i]-ave; s2+=d*d; }
cout<<(s2/(nA-1))<<"\n";
// 2D input (teacher style, 1-based with padding)
int i,j; double B[3+1][3+1];
cout<<"\ninput 3x3 matrix B (row by row) ?\n";
// prompt
for(i=1;i<=3;i++){
    for(j=1;j<=3;j++){
        cin>>B[i][j]; // user types 9 numbers
    }
}
```

// If / else if / else ladder template

```
if(x<0){ cout<<"neg\n"; }
else if(x==0){ cout<<"zero\n"; }
else{ cout<<"pos\n"; }
```

// while + break example (compact)

```
int i=1; double x=40.0,z=0.0;
while(x>1){
    i++; x=x/2;
    if(i>3){ break; }
    else if((x<10)&&(i>1)){ z=1.5;
cout<<"nz1"; }
    else{ z=-1.5; cout<<"nz2"; } }
A[i]=x+z; }    TRANSPOSE: int i,j; double
```

```
B[3+1][3+1],C[3+1][3+1]; // declare arrays (1-
based padding)
cout<<"\ninput 3x3 matrix B (row by row)?\n";
for(i=1;i<=3;i++){ for(j=1;j<=3;j++){
    cin>>B[i][j]; } } // read 9 values
```

```
for(i=1;i<=3;i++){ for(j=1;j<=3;j++){
    C[j][i]=B[i][j]; } } // transpose B->C
cout<<"\nC = transpose(B):\n"; // label
for(i=1;i<=3;i++){ for(j=1;j<=3;j++){
    cout<<C[i][j]<<" ";
    }
}
```

Mock exam

Summary

Quiz

Ask a question about this document